

AI Engineering Lab

Chair of AI Engineering

SoSe 2026

1 Overview

In this lab, you build an agentic system that performs a set of defined tasks. First, you review the state of the art of agent harnesses: what their design principles are, how they orchestrate several components, and how they can be equipped with monitoring/observability to track their internal workings and detect root causes of failures. Then, you implement a first version of a harness that, together with the provided LLM, users can interact with via terminal to solve tasks that require navigating a local file system. Subsequently, you extend the harness to support long-term memory, skills, and access to new types of tools. Finally, you fortify the harness with prevention of sensitive or consequential operations, hooks for monitoring/observability, and support for specialized subagents.

The learning objectives are:

- Know what the components of an agentic system are, how they work, and the role they play.
- Understand how these components interact with each other and how these interactions extend the capabilities of an agentic system.
- Understand how to ensure the quality of the individual components in an agentic system as well as for the overall system.
- Be able to implement an agent harness, including its source code and tests.
- Extend the agent harness with telemetry data via hooks for monitoring/observability purposes.

Important: The goal of this lab is not to implement a high-quality agent harness with many features. This is important but secondary. The goal is, rather, to gain an in-depth understanding of these systems through a guided implementation as a teaching method.

2 Background

Large Language Models (LLMs) and AI agents are revolutionizing an ever-increasing number of industries. In their basic form, LLMs are conversational interfaces with no access to external information other than that provided by a user during a conversation. In contrast, agents are LLMs running in a loop, autonomously using tools that enable interaction with an environment such as searching the internet, navigating a file system, or querying an API. These tools are necessary to perform tasks that an LLM could not accomplish on its own, such as booking a flight, solving a GitHub issue, or scheduling a meeting. There are different approaches to building such a loop, but one of the most popular is *ReAct* [1]: given a task prompted by a user, the LLM iteratively reasons about the next action toward solving it and decides whether that action requires one of the available tools. If a tool is called, its output is appended to the context and serves as evidence of the task progress on the following LLM call; if no further action is needed or possible, the LLM responds to the user, ending the loop (see Figure 1).

Agents vary widely in the kind of tasks they can perform, the environment in which they act, and the safety and security implications of their actions, among other aspects. Handling this variation reliably motivates a

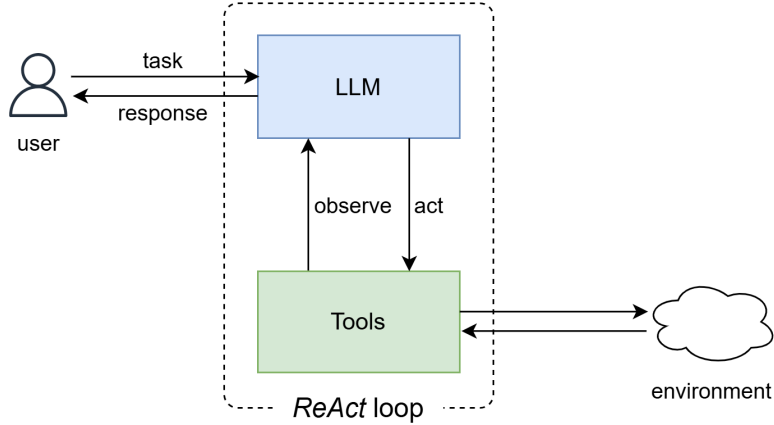


Figure 1: The *ReAct* approach.

dedicated infrastructure layer, now typically known as a harness, which orchestrates different components to turn a stateless LLM into a functional, extensible, and reliable agent. In these terms, an agentic system can be understood as the composition of an LLM and a harness, and its components are summarized as follows:

- **LLM:** The core reasoning and generation engine, stateless across calls, that decomposes tasks, selects the next action, interprets user feedback and tool outputs, and decides when to revise or respond to the user. A sufficient level of instruction-following capability is expected to reliably comply with user guidance and system rules; beyond that threshold, the widest optimization window lies not in the LLM itself but in how the surrounding harness curates its context.
- **Loop:** The control structure that drives the agent, expressed as a state machine that alternates between a reasoning state (an LLM call) and an acting state (a tool call). The LLM output decides each transition: a tool call moves the machine to the acting state, whose observation is appended to the context before returning to reasoning, while a final answer moves it to a terminal state that ends the run. Guard conditions such as iteration caps, token or time budgets, and error handling bound the machine so it always terminates, preventing runaway execution when the LLM neither returns a final answer nor a valid action.

Up-front planning: A typical loop such as *ReAct* can be extended by instructing the agent to plan the task up front, then the loop needs to contrast task progress against this initial plan. Although a user can explicitly ask for it, some harness implementations include equivalent instructions into the system prompt, thus any agent run invests its first reasoning effort on creating this plan.

- **Context:** The short-term memory, updated after every loop step, that includes system instructions, conversation history, tool schemas, and outputs. Because LLM performance degrades as the window fills (an effect known as context rot), effective context engineering seeks the smallest set of high-signal tokens that still yields the desired behavior. As runs grow long, the harness manages this budget through techniques such as compaction (summarizing older turns) and offloading state to external memory, trading some fidelity for sustained coherence and lower cost [2].
- **Tools:** Mechanisms that enable agents to search the internet, navigate a file system, query an API, or run CLI commands. Tools expose deterministic, schema-defined interfaces, even when their outputs are environment-dependent; and although they are prone to failures, these must be communicated upstream effectively. Because tools define the contract between the agent and its action space, they should be token-efficient and unambiguous: a bloated set with overlapping functions makes it as hard for the agent to choose correctly. MCP [3] has been popularized as a standard for the definition of tools.
- **Skills:** Modular, loadable bundles of instructions (optionally scripts, references, or examples) that package reusable know-how for a class of tasks, such as filling a PDF form or querying a specific API.

Rather than holding every skill in context at all times, the harness discloses it only when the task calls for it using a just-in-time mechanism. Because skills are portable text, they can be shared, versioned, and composed across agents, and a well-stocked skill library can sometimes stand in for explicit multi-agent coordination.

- **Long-term memory:** Persistent information retained across runs, such as user preferences, learned facts, or summaries of previous conversations. It lives outside the context and is consulted on demand. At writing time, the harness must decide what is worth persisting and when. At reading time, retrieval combines complementary strategies: dropping small, high-value files into the context up front, and keeping lightweight identifiers, such as file paths, that let the agent load information just-in-time. This just-in-time mechanism is similar to the one used for skills but carrying declarative facts (what happened) rather than procedural knowledge (how to do something).
- **Hooks:** Custom functions triggered at specific points in the agent lifecycle, such as before a tool call or after an LLM response. Pre-use hooks can validate arguments, enforce permission policies, or block risky commands, while post-use hooks can sanitize outputs, compact logs, update memory, or trigger follow-up verification. By making these control points explicit, hooks turn a raw, LLM-selected action into a monitored transition in the loop and provide the natural insertion point for logging and observability.
- **Permission management:** The policies and controls that determine the actions and resources an agent may access, often requiring user approval for sensitive or consequential operations. Some harnesses follow a multi-tier model separating low-risk observation from high-risk action: a read-only tier for browsing and inspection, a sandbox-edit tier for local changes and test runs, and a full-access tier for irreversible or outward-facing operations. Actions in the last tier are typically guarded by mandatory human-in-the-loop gates, since their consequences can extend beyond the sandbox.
- **Sandboxing:** Isolation of agent-executed code and tools within a restricted environment to limit access to the host system, network, files, and other sensitive resources. This can be realized at different levels of abstraction, from operating system primitives to containers or microVM/WASM isolation, up to a fully separated operating system [4]. Beyond containment, a stable sandbox improves reproducibility, since an agent can replay the same command, seed, or dependency set under comparable conditions, ensuring a failure reflects a genuine defect rather than environment drift.
- **Agent-to-agent communication:** The protocols and mechanisms through which agents exchange tasks, context, intermediate results, and status information to coordinate their work. **Specialized subagents** are typical cases of agent-to-agent communication: a main agent delegates a subtask to a specialized one, providing only the relevant information and a schema of the expected result. Because the subagent works in its own clean context window and returns only a condensed summary, detailed intermediate work stays isolated from the main agent, keeping the coordinating context focused as the system scales.

Most of these components are domain-agnostic: they are defined independently of what an agent running on a harness produces, whether natural language text, code, or images. What mainly specializes a harness to a domain are the concrete tools and skills plugged into it, along with the sensors used to verify outputs. Running a test suite for code, checking layout and consistency properties for a slide deck, or reconciling figures for a report are all instances of the same generate-then-verify pattern.

Recent harness developments have been framed in several complementary ways. One perspective centers code as the execution medium, where reasoning, acting, and environment state become executable, inspectable, and verifiable artifacts [5]; this view is prominent partly because code is the domain where verification is cheapest and most deterministic (a test passes or fails), whereas other domains often fall back to LLM- or human-judged criteria. A second decomposes the harness into functional modules, such as perception, memory, and reasoning, and studies the contribution of each module empirically, finding that different modules dominate for different types of tasks [6]. A third draws a boundary between the LLM and the infrastructure around it, and treats that infrastructure, rather than the LLM, as the primary lever for agent performance and the first place to look when an agent fails [7]. Across these perspectives, a recurring challenge is that harnesses remain difficult to monitor, compare, transfer, and ablate as objects in their own right [8].

3 Deadlines Overview

- Registration: Tuesday morning, September 1nd
- Individual daily reports: Monday - Thursday
 - Submissions to the GitLab repository assigned to the group by the end of the day (see Section 5.1).
- Group presentations: Friday, September 4th and 11th
 - Slide submission **as PDF** before 9am on Friday via email to `lukas.schulte@uni-passau.de`, further updates are not allowed.
- Individual presentations: Friday, September 18th
 - Slide submission **as PDF** before 9am on Friday via email to `lukas.schulte@uni-passau.de`, further updates are not allowed.
 - Time slots will be assigned in advance.
- Individual written reports: Sunday, September 27th
 - Slide submission **as PDF** before midnight via email to `lukas.schulte@uni-passau.de`.
- Individual oral exam: Wednesday, September 30th
 - Time slots will be assigned in advance.

4 Schedule

The lab is divided into three sprints, one per week. You will receive updated instructions on a weekly basis, including information on the minimum requirements for each deliverable.

4.1 Monday - Wednesday

You work on the minimal requirements of the agent harness that you are requested to build, as well as further self-set goals that you discuss opportunely with the teaching assistants (TAs) present in the lab. While attendance during these days is not mandatory, you are required to write individual daily reports. These must be committed to the GitLab repository by the end of each day to be reviewed by the TAs the next morning. Students who miss the deadline for daily reports three times will fail the course. For more information, see Section 5.1.

4.2 Thursday

Like in previous days, you continue to work on your agent harness, while the TAs may be interested in reviewing your progress. For this, your team needs to have a runnable and documented version committed to the GitLab repository by 11am, either on a ‘stable’ branch or as a version tag on the ‘main’ branch. Your team will then receive feedback the same day. The whole team should be present.

4.3 Friday

You or your team, depending on the week, presents the current status of your agent harness in front of the class. Notice that whereas the individual presentation is graded, the group presentations are not; however, all team members must be present during all presentations and, for group presentations, are required to have equal speaking time. A presentation is aimed to 10-20 minutes length plus an additional time for questions, but this might vary depending on the number of teams. The final length will be announced after the registration period ends.

5 Grading

Your final grade consists of four parts:

- Daily reports
- Final presentation
- Written report
- Oral exam

To receive a final passing grade, you need to sufficiently complete all parts. The examination for each part will evaluate your ability to explain how certain components were implemented, the architecture decisions you made, and how thoroughly you designed the quality assurance and observability measures.

5.1 Daily Reports

The individual daily reports must be committed to the `daily_reports` folder of the GitLab repository assigned to your group by the end of each day (Monday - Thursday) and will be checked by the TAs the next morning. Students missing the deadline for the daily reports three times will fail the course. You must use Markdown and adopt the general naming scheme:

```
{YourName}-{WeekX}-{DayOfWeek}.md
```

You must keep the report short, including at least two sections: one for your individual contributions, and one for team related work. An example can be found below:

```
daily_reports/LukasSchulte_Week1_Tuesday.md

# My contributions:
- impl: LLM API timeout handling
- test: unit tests for ToolRegistry (empty dirs, perm errors)
- review: code review for Fabian Peña (impl: base functionality for terminal UI)

# Team discussions and decisions:
- Discussed adding feature ABC, decided against.
- Asked Fabian Peña and Arthur Dent for review (impl: LLM API timeout handling)
```

5.2 Individual presentation

During the final presentation, you must individually present one aspect of the agent harness your team implemented. This must include both background (a presentation) as well as a demo (live or pre-recorded with live commentary). The aspect that you must present and the time slot will be assigned during the third week.

5.3 Individual written report

You must also individually write a technical report about the same aspect that was assigned to you for the presentation. The length of the report must be between 3.000 and 4.000 words, excluding tables, graphics, and appendices.

5.4 Individual oral exam

You must partake in an oral exam of 10 minutes in which you will be asked questions regarding the understanding AI agents in general, as well as specifics of the agent harness your team implemented.

Minimum requirements

These deliverables define the minimum mandatory requirements for passing this lab. Fulfilling them qualifies you for a passing grade (4.0) at most. To achieve a higher grade, you are expected to independently identify, design, and implement additional features or improvements to your agent harness beyond the scope of the weekly assignments, as well as understand how they work. The scope, quality, and technical depth of these self-directed extensions will be assessed across all grading components.

6 Lab Policies

You are expected to reach the goal of this lab by using modern software engineering practices, including collaborative teamwork and the deliberate use of AI assistance. Concretely, this means:

- **Shared code ownership:** All team members are responsible for the entire codebase, not just their individual contributions. Practices such as regular team meetings, code reviews, and pair programming may help you achieve this.
- **AI usage:** You are encouraged to use AI tools (e.g., LLMs, code assistants) to support your work, including brainstorming, implementation, debugging, and documentation. However, you must understand and be able to explain all components of your code and the decisions that were made, regardless of whether it was AI-assisted.

The oral exam may contain questions about the entire source code; progress and team work will be checked through the daily reports.

7 General Requirements

7.1 Implementation target

You are required to implement your agent harness as a **terminal-based (CLI) application**. No graphical user interface (GUI) or web-based frontend is expected or desired. Configuration must be persistent, for example through JSON or YAML files.

7.2 Programming Language & Version

All code must be written in **Python 3.12**.

7.3 Containerization

Your application must be delivered and executable inside a **container** (Podman or Docker). The base image must be `python:3.12-slim-bookworm` (Debian based). The containerization serves a dual purpose:

1. **Security:** It provides a sandboxed environment that isolates agent-executed code and local tools from the host system.
2. **Reproducibility:** The TAs will test your application independently on different machines. A container ensures a consistent environment (same OS, same dependencies, same configuration) so that any failure reflects a genuine defect rather than an environment drift. The container configuration files (`dockerfile/containerfile`, `compose.yaml`, etc) must be fully committed to the GitLab repository (**excluding** any authentication keys, logins, etc.) and sufficiently documented.

The container must be self-contained: all dependencies must be installed within the image, and the agent must be runnable via a single `podman run` (or `docker run`) command. Similarly to running a command like `claude`, the terminal must enter agent mode so that the user can begin interacting with it.

7.4 LLM access

You are provided with access to the InnKube cluster for LLM inference. Further details on the specific LLMs and endpoint are provided in Section 8. Usage of external APIs (e.g., OpenAI, Anthropic) is allowed, but all tasks will be tested using the LLMs provided through the InnKube API.

7.5 Code Quality & Maintainability

- Use version control (Git) and commit early and often.
- Write modular, well-structured code with meaningful function and variable names.
- Include docstrings and inline comments where necessary.
- Provide a `README.md` with setup instructions, usage examples, and an overview of your architecture.

7.6 Testing

Your application must be testable. You are expected to include automated tests (e.g., `pytest`) as part of your deliverables. Further specifications for testing will be detailed in Section 8.

8 Weekly Targets

8.1 Preparations

On the first day, form teams of exactly three members, choose one member to send an email to the TAs (email: `lukas.schulte@uni-passau.de`) with the following information:

- Full names
- FIM¹ (not ZIM) account IDs

You must contact the TAs if, at the end of the day, you are still looking for a team or an additional team member. Team composition is finalized once the registration deadline is over to account for late registrations and drop-outs. However, to avoid losing time, it is recommended to start working from day one.

Once you have notified the TAs about the composition of your team, you will be assigned to a GitLab repository. This is your main team workspace and will be used by the TAs to check in on your work. Keep it tidy and up to date.

You will also receive an access token for the InnKube LLM inference endpoint² where different medium-sized LLMs are available. This access token is valid during the course of the lab (expiring on September 30th). We recommend using `qwen-agentworld-35b-a3b` or `gemma4-31b-it` for agent running.

8.2 Week 1

The overall goal of this week is to build an agent (running on top of your harness) with access to a local folder where it can navigate and modify files. Before starting, familiarize yourself with the typical components of an agentic system (see Section 2), with a focus on those required to build the agent described before. Draw a diagram that represents how the selected components interact with each other and establish an implementation plan for this week. Discuss your ideas with the TAs and commit the generated resources to GitLab.

The minimum deliverables for this week are:

- A diagram that represents the first version of your agent harness.
- Implementation of the core *ReAct* loop.

¹<https://www.fim.uni-passau.de/en/it-services/login-and-account/fim-accounts/>

²https://innkube.pages.gitlab.innkube.fim.uni-passau.de/documentation/our_infrastructure/llms-hub.html

- Experiment first with a simple prompt-reason loop, i.e., a multi-turn chat with no access to external data.
- Implementation of a terminal-based application with basic functionality.
- Implementation of a JSON- or YAML-based configuration mechanism to set parameters such as access token, LLM, and temperature.
 - The access token must not be committed to the repository. Keep it as environment variable.
- Containerization via Podman or Docker, establishing a basic security boundary around the agent’s file system access.
- Implementation of a tool registry with the following capabilities while enforcing basic security measures:
 - The agent can create folders and files
 - The agent can navigate folders and search in files
 - The agent can read and modify files
 - The agent CANNOT delete folders or files
- Implementation of a maintainable test suite that covers the previous features.
 - Keep the test suite up to date and runnable throughout the lab.
 - Add multiple unit tests as well as end-to-end tests for the file system access capabilities (see examples below).

Remember that you are expected to find further deliverables to achieve a better grade. Make sure not only to pre-solve next week deliverables.

E2E test case examples, your tests must cover realistic use cases

numbers_in_words

Tools exercised: `create_folder`, `write_file`, `search_text`.

Prompt: Create a folder called `output` and inside it a file called `numbers.md` that contains the a list with the numbers from 1 to 20 in words. At least, make sure the first and last numbers exist.

Assert:

- Folder `output/` exists.
- File `output/numbers.md` exists and contains the byte-exact texts ‘one’ and ‘twenty’.
- Final agent turn acknowledges.

find_and_inspect

Tools exercised: `list_files`, `read_file`.

Setup: Pre-populate the following tree before running the test:

```
path_ABC/  
├── data/  
│   ├── notes.txt → "This\nfile\nhas\nnone\nline"  
│   └── readme.md → "Kartoffelsalat"  
└── empty/
```

Prompt: Tell me what is inside `path_ABC/`. If `notes.txt` exists, read it and tell me how many lines it has.

Assert:

- Assistant mentions both `notes.txt` and `readme.md` in the listing.
- Assistant reports 5 lines inside `notes.txt`.

8.3 Week 2

Building on Week 1, extend your harness with support for long-term memory, and more tools accessed through MCP servers. First, revisit your system-architecture diagram and update it where necessary to evidence how these new components interact with your existing harness. Discuss significant architectural changes with the TAs and commit the revised diagrams to GitLab.

The minimum deliverables for this week are:

- Design, implement, and test a configurable integration for MCP servers. The integration should:
 - Connect to at least one configured MCP server.
 - Discover available tools at runtime.
 - Make the discovered tools available to the LLM.
 - Invoke the selected tool and append its result to the context.
 - Handle connection and tool-execution errors in a controlled manner.
- Design, implement, and test an extension to an existing MCP server that exposes at least one new tool.
 - The new tool must expose a capability that was not previously available through the selected MCP server.
 - You do not have to implement the underlying capability from scratch. For instance, the tool may wrap an existing command-line application, library, REST API, local service, or another suitable system.
 - You are responsible for designing and implementing the MCP interface, including the tool's name, description, input schema, argument validation, result transformation, error handling, and tests
 - The tool must address a non-trivial use case unrelated to programming or file system manipulation.
 - Generic shell-execution, arbitrary-code-execution, and unrestricted HTTP-request tools do not satisfy this requirement. The tool must expose a bounded, clearly described capability.
- Design, implement, and test a persistent long-term memory component.
 - The agent must be able to store information during one session and retrieve relevant information in a later session, even after the application restarts.
 - The mechanism to decide what and when to store and retrieve information from the long-term memory must be clearly stated.

- Extend and maintain the test suite from Week 1.
 - Add tests for the MCP integration, the new MCP tool, and the long-term memory.
 - Add at least one end-to-end test covering runtime MCP tool discovery, invocation, and result handling.
 - Ensure that the Week 1 functionalities continue to work.

As in Week 1, these are the minimum requirements. Further extensions may improve the grade, but should deepen the current scope rather than pre-solve next week deliverables.

8.4 Week 3

Building on Weeks 1 and 2, extend your harness with a monitoring/observability layer and support for permission management and specialized sub-agents. These features should apply consistently across the system rather than being implemented as isolated special cases (e.g., a sub-agent must run on the same harness that its parent agent). Don't forget also to update your system-architecture diagram and discuss significant changes with the TAs.

- Enable monitoring of agent runs, LLM calls, input and output tokens, memory operations, tool calls, among others relevant metrics.
 - Collect metrics through reusable lifecycle hooks or an equivalent decoupled instrumentation mechanism.
 - Instrument the application using the Prometheus client library and expose the collected metrics through a Prometheus-compatible metrics endpoint.
 - Configure a Prometheus server to scrape and store the application's metrics.
 - Provide a dashboard that queries Prometheus. The choice of dashboard software is left to you. The dashboard must show at least:
 - * Agent throughput and the number of active runs
 - * Agent-run and component latency
 - * Success and failure rates
 - * LLM usage, including token consumption where available
 - * Built-in and MCP tool usage
 - * Permission decisions
 - * Sub-agents activity
 - In addition to aggregate metrics, provide structured logs or traces that allow the execution of an individual agent run to be inspected.
- Design, implement, and test a permission management mechanism for built-in and MCP tools.
 - Policies must support at least the outcomes allow, deny, and require-user-confirmation.
- Design, implement, and test specialized sub-agents running into the same harness.
 - A sub-agent must be a separately agent instance with its own role or instructions, context, execution limits, and permissions.
 - The parent agent must be able to delegate a task to a sub-agent and receive a result that can be incorporated into the parent context.
 - Demonstrate at least two specialized sub-agent configurations and at least one task that benefits from delegation.
 - Record properly sub-agent executions as part of the corresponding parent run in the observability data.

- Extend and maintain the test suite from the previous weeks.
 - Test monitoring, permission management, sub-agent delegation, and their interactions.
 - Add at least one end-to-end test covering sub-agent delegation, permission enforcement, tool execution, and observability.

References

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *The Eleventh International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=WE_vluYUL-X
- [2] P. Rajasekaran, E. Dixon, C. Ryan, and J. Hadfield. (2025, Sep.) Effective context engineering for AI agents. Anthropic. [Online]. Available: <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- [3] Model Context Protocol. What is the model context protocol (MCP)? Model Context Protocol. [Online]. Available: <https://modelcontextprotocol.io/docs/getting-started/intro>
- [4] Anthropic. Choose a sandbox environment. Claude Code Documentation. [Online]. Available: <https://code.claude.com/docs/en/sandbox-environments>
- [5] X. Ning, K. Tieu, D. Fu, T. Wei, Z. Li, Y. Bei, J. Zou, M. Ai, Z. Liu, T.-W. Li, L. Chen, Y. Zhao, K. Yang, B. Li, C. Qian, G. Li, X. Lin, Z. Zeng, R. Qiu, S. Chen, Y. Sun, X. Yang, R. Wang, R. Pan, C. Yang, D. Zhang, L. Fang, Z. Cui, Y. Cao, P. Chen, D. Sun, R. Chen, M. Srinivasan, N. Mathur, Y. Xia, H. Li, H. Yan, P. Lu, L. Zhang, T. Zhang, H. Tong, and J. He, “Code as agent harness,” 2026. [Online]. Available: <https://arxiv.org/abs/2605.18747>
- [6] Y. Zhang, H. Yu, L. Hu, H. Jin, and H. Zhang, “General modular harness for LLM agents in multi-turn gaming environments,” in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025. [Online]. Available: <https://arxiv.org/abs/2507.11633>
- [7] V. Trivedy. (2026, Mar.) The anatomy of an agent harness. LangChain. [Online]. Available: <https://www.langchain.com/blog/the-anatomy-of-an-agent-harness>
- [8] L. Pan, L. Zou, S. Guo, J. Ni, and H.-T. Zheng, “Natural-language agent harnesses,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.25723>